

Abstract class and Abstract method

- ❖ Abstract is a keyword
- ❖ Abstract keyword can be used in two ways:
 - ⇒ With classes
 - ⇒ With methods
 - ⇒ Abstract keyword cannot be used for variable.
- ❖ Classes defined without abstract modifier are called as **concrete class**. Concrete class can be instantiated (creating the object).
Example: - class person { }
- ❖ Classes defined with abstract modifier are called as **Abstract class**. Concrete class cannot be instantiated but can be subclass.
Example: - abstract class person { }
- ❖ Method defined without abstract modifier is called as **concrete Method**. Concrete method must have body.
Example: - void sleeping () { }
- ❖ Method defined with abstract modifier is called as **Abstract Method**. Abstract method must not have body.
Example: - abstract void sleeping ()
- ❖ Abstract class can be used when you want to define default behaviour for sub classes.
- ❖ You cannot use following modifier with abstract method.

1. private	2. static	3. Final
4. strictfp	5. synchronized	6. native

You can check this combination with any program.

Ex1.java

```
abstract class Ex1{
private abstract void m1();
}
// error: illegal combination of modifiers:
abstract and private
```

Ex2.java

```
abstract class Ex2{
static abstract void m1();
}
// error: illegal combination of modifiers:
abstract and static
```

Ex2.java

```
abstract class Ex3{
final abstract void m1();
}
// error: illegal combination of modifiers:
abstract and final
```

Ex4.java

```
abstract class Ex4{
synchronized abstract void m1();
}
// error: illegal combination of modifiers:
abstract or synchronized.
```

Ex5.java

```
abstract class Ex5{
strictfp abstract void m1();
}
// error: illegal combination of modifiers:
Abstract and strictfp.
```

Ex6.java

```
abstract class Ex6{
native abstract void m1();
}
// error: illegal combination of modifiers:
abstract and native.
```

Ex7.java

```
final abstract class Ex7{
abstract void m1();
} //error: class Ex1 can be either
abstract or final, not both
```

Ex8.java

```
abstract class Ex8{
protected abstract void m1();
} //you can use protected modifiers
with abstract.
```

Rules: -

1. When you don't know the implementation of method then you can avoid the body of method.
2. When you are not writing body of method then that method must be declared as abstract.
3. When you want to force the subclasses to provide their own implementation for some method then that method can be declared as abstract.
4. When a class contains one or more abstract method then the class must be declared as abstract but the reverse is not true i.e. we can have an abstract class without having any abstract method also.
5. When you do not want to instantiate the class then you can declare that class as abstract.
6. When a class is extending abstract class then that class has to override all the abstract method of that abstract super class.
7. When subclass is not overriding one or more abstract method then subclass must be declared as abstract.
8. Abstract class cannot be instantiated i.e. you cannot create the object of abstract class but you can declare reference variable.
9. Abstract class can contain all the members that can be written in concrete class.
10. Static member of an abstract class will be accessed with class name.
11. Instance member of an abstract class will be accessed with sub class object.
12. When you create the object of subclass then instance block and constructor of abstract class will be called.
13. When abstract class is public then abstract methods of abstract class should be either protected or public.

Person.java

```
Package com.p1;
public abstract class Person {
abstract void sleeping ();
}
```

Student.java

```
Package com.p1;
import com.p1.*;
public class Student extends Person {
void sleeping(){
//public void sleeping(){
System.out.println("Stud =>sleeping");
}
```

Programs:-

OOPS392.java

```

class Person{
void sleeping(){ }
}
class Student extends Person{ }

class OOPS392 {
public static void main(String args[]){
Person pob=new Student();
pob. sleeping();
}
}

```

OOPS393.java

```

class Person{
void sleeping();
}
class Student extends Person{ }

class OOPS393 {
public static void main(String args[]){
Person pob=new Student();
pob. sleeping();
}
}

```

OOPS394.java

```

class Person{
abstract void sleeping();
}
class Student extends Person{}

class OOPS394{
public static void main(String args[]){
Person pob=new Student();
pob. sleeping();
}
}

```

OOPS395.java

```

abstract class Person{
abstract void sleeping();
}
class Student extends Person{}

class OOPS395{
public static void main(String args[]){
Person pob=new Student();
pob. sleeping();
}
}

```

OOPS396.java

```

abstract class Person{
abstract void sleeping();
abstract void walking();
}
class Student extends Person{
void sleeping(){
System.out.println("Stud=>sleeping()");
}
void walking(){
System.out.println("Stud=>walking()");
}
}
class OOPS396{
public static void main(String args[]){
//Person pob=new Person();
Person pob=new Student();
pob. sleeping();
pob. walking();
}
}

```

OOPS397.java

```

abstract class Person{
abstract void sleeping();
abstract void walking();
}
abstract class Student extends Person{
void sleeping(){
System.out.println("Stud=>sleeping()");
}
}
class CurStudent extends Student{
void walking(){
System.out.println("CurStud=>walking()");
}
}
class OOPS397{
public static void main(String args[]){
//Person pob=new Person();
Person pob=new CurStudent();
pob. sleeping();
pob. walking();
}
}

```

OOPS398.java

```

abstract class Person{
abstract void sleeping();
}
abstract class Student extends
Person{
void sleeping()
{
System.out.println("Stud=>sleeping()");
}}
class CurStudent extends Student{}
class OOPS398{
public static void main(String args[]){
Person pob=new CurStudent();
pob. sleeping();
}}

```

OOPS399.java

```

abstract class Person{
public abstract void sleeping();
}
class Student extends Person{
void sleeping()
{
System.out.println("Stud=>sleeping()");
}}
class OOPS399{
public static void main(String args[]){
Student pob=new Student();
pob. sleeping();
}}

```

OOPS400.java

```

abstract class Hello{
static int a=1234;
static void show()
{
System.out.println("Hello=>show()");
}
}
class OOPS400{
public static void main(String args[]){
System.out.println(Hello.a);
Hello.show();
}
}

```

OOPS401.java

```

abstract class Hello{
int a;
Hello (int a){
this. a=a;
System.out.println("Hello (int) cons");
}
{
System.out.println("Hello=> I.B");
}
void show(){
System.out.println("Hello =>show()");
}}
class Hai extends Hello{
Hai(int a){
super(a);
}}
class OOPS401{
public static void main(String args[]){
Hai hai=new Hai(10);
hai. show();
}
}

```

Write your note here.....

Program output /error and details:-

Prog No.	output	error and details
OOPS392	[Blank]	Here, Person is concrete class and When you don't know the implementation of method then you can avoid the body of method.
OOPS393	Compilation error	[Error: This method requires a body instead of a semicolon] most use body notation or declare as abstract method.
OOPS394	Compilation error	[Error: The type Person must be an abstract class to define abstract methods. The abstract method sleeping in type Person can only be defined by an abstract class]
OOPS395	Compilation error	[Error: The type Person must be an abstract class to define abstract methods. The abstract method sleeping in type Person can only be defined by an abstract class]
OOPS396	stud=>reading() stud=>working()	Refer page 142 point 4
OOPS397	Stud-sleeping() CurStud-walking()	Refer page 142 point 6
OOPS398	Stud=>sleeping()	Refer page 142 point 7
OOPS399	Compilation error	[Error: Cannot reduce the visibility of the inherited method from Person]Here, we are attempting to access weaker access modifier.
OOPS400	1234 Hello=>show()	Refer page 142 point 8, 9 and 10.
OOPS401	Hello => I.B Hello(int)cons Hello =>show()	Refer page 142 point 11 and 12.

Real time Example: TV Remote and mail etc.

Remote is a interface between user and TV. Which has buttons like 0 to 10, on and off etc. But we don't know circuits inside remote.

When you are open the mail then it will display some functional button we are click on the button and use the function but you don't know what the action performed internally.